Summary of Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding (Wu et al. 2026)

Shruti Jayaraman (shrujaya), Pranav Mahalingam (mpranavm), Srikrishnan Ravichandran (srikrish), Nikhil Janarthan Surendran (nikhilnj)

Problem and Motivation

Diffusion based Large Language Models (D-LLMs) are a new generation of LLMs that generate sequence data by iteratively refining noisy, masked content into coherent content, rather than just sequentially predicting tokens in one direction of a sequence like autoregressive (AR) models. D-LLMs have gained significant traction in recent times due to **parallel token generation**, **more effective use of global context** and **iterative refinement**. However, D-LLMs have pitfalls in training stability, hyperparameter sensitivity and from a systems perspective, they have a **throughput gap** and are **slower** when compared to AR models.

The authors attribute this gap to two main issues in D-LLMs:
(1) D-LLMs do not support key-value (KV) caching unlike AR models. The **bidirectional attention mechanism breaks cache validity.** In D-LLMs, each token attends to all others in the sequence at every step. As the model updates masked tokens, the keys and values of all tokens change, requiring a complete re-computation of the attention mechanism at every denoising step. Secondly, the **non-sequential decoding** caused by D-LLMs adopting flexible generation orders, results in the model not knowing beforehand which KV pairs need to be cached or updated. Finally, the cache becomes **stale** since a cached KV-pair based on a noisier version of a token becomes useless after that token is refined at a later step.

(2) The generation quality tends to degrade when decoding multiple steps in parallel. While diffusion models can theoretically predict multiple tokens at once, doing so causes a significant drop in generation quality. The authors trace this to a "conditional independence assumption". When the model samples interdependent tokens simultaneously without accounting for their relationship to one another, it disrupts critical dependencies and leads to incoherent outputs.

The two above issues serve as motivation to overcome the slow inference speed of D-LLMs without requiring expensive retraining. The authors tackle the issues by proposing (1) A **Block-wise Approximate KV Cache** and (2) **Confidence-Aware Parallel Decoding.** The authors report up to 27.6x throughput gains with minimal loss in accuracy.

Related Works

Diffusion Models: Diffusion models have emerged as a transformative paradigm in generative modelling, initially achieving remarkable success in continuous domains such as image and audio synthesis before expanding into natural language processing. Diffusion LLMs in the forward process, apply noising and in the reverse process, turn random noise (or masked tokens) into coherent text through a gradual denoising process. **Masked Diffusion Models (MDMs)** progressively replace text tokens with a [MASK] token during a forward process, and learn to reverse this process to generate text. The reverse process is computationally inefficient for generation as it modifies only one token per step. A common work-around for this is **τ-leaping.**

τ-leaping is a sampling acceleration technique that simulates multiple denoising steps and effectively takes a "leap" of τ denoising steps in a single leap, thereby applying a significant denoising step at once instead of updating every minor change in the probability of a token being a specific word. As explained in the previous section, the authors note that the τ-leaping generates multiple tokens in parallel operating under a conditional independence assumption which is not true.

The paper's baselines, **LLaDA** (Large Language Diffusion with Masking) is a standard diffusion LLM that treats tokens uniformly, allowing for bidirectional context awareness. **Dream** (Diffusion REAsoning Model) specifically focuses on reasoning tasks which uses "any-order" generation, meaning it can fill in gaps in any sequence order, enhancing flexibility and planning.

LLM Acceleration: Key-Value (KV) Cache is a fundamental optimization technique in modern LLM inference which enables efficient autoregressive text generation by storing and reusing previously computed attention states. However, as explained above, adopting them to D-LLMs pose significant challenges (see point (1) in the Problem and Motivation). **Block Diffusion** overcomes this key limitation by breaking text generation into chunks (blocks), where each block is generated via a discrete diffusion process (simultaneous token refinement) while conditioning on previous blocks. This is a hybrid approach that uses the structure of AR models to generate blocks sequentially but denoises tokens within a block in parallel. This enables the usage of a block-level "approximate" KV cache that the authors propose where the KV states of completed, previous blocks are stored and reused, significantly reducing repetitive calculations.

Non-Autoregressive (NAR) Generation: NAR methods shift from sequential token generation to generating multiple tokens simultaneously to speed up inference. However, this speed advantage often comes at the cost of generation quality as explained previously.

Solution Overview

1. Key-Value Cache for Block-Wise Decoding

Approximate block-wise KV cache**:** Fast-dLLM adapts KV-caching to MDMs by decoding in blocks and reusing stored key/value activations across multiple inference steps inside a block. Because masked diffusion models use bidirectional attention, a perfect incremental cache (as in autoregressive decoders) is

not strictly correct as the cache would need to be continuously updated, making the cache "approximate". Fast-dLLM therefore uses a **block-wise approximate** cache: it computes KV activations for the prompt (and for other blocks as needed), reuses those cached activations for several decoding iterations inside the block, and only refreshes/recomputes the cache after a block completes (or periodically). The approximation is justified empirically where the authors show KV activations between adjacent inference steps are highly similar, so reusing previous KVs induces only negligible error while saving a lot of repeated attention computation. This design yields large throughput gains with minimal accuracy drop and avoids extra model training or architectural change.

DualCache (bidirectional cache): DualCache extends the above idea by caching keys and values for both the prefix (the already-fixed prompt & decoded tokens) and the suffix (the still-masked tokens in the block). Under the block decoding schedule, the suffix is a known mask pattern, so the model's KV activations for masked suffix positions are similarly stable across nearby steps; caching them provides additional reuse. In practice DualCache yields larger end-to-end speedups (especially for longer generation lengths and larger prefill contexts) while maintaining competitive accuracy. The authors present ablations showing DualCache outperforms prefix-only caching in throughput with only small accuracy trade-offs.

2. Confidence-Based Parallel Decoding

Threshold strategy (confidence gating): Fast-dLLM addresses the second issue described above by computing a per-token **confidence score** (e.g., the max softmax probability) at every iteration and only unmasking tokens whose confidence exceeds a global threshold. If none pass it, the single most confident token is unmasked to guarantee progress. The theoretical rationale is that when each selected token's marginal probability for its chosen symbol is very high (i.e., high-confidence regime), the product-of-marginals approximation is close to the true joint and greedy parallel decoding matches greedy sequential decoding (formalized in their Theorem 1). In experiments the authors sweep the threshold in [0.5,1.0] and commonly use 0.9 as a practical trade-off between safety and parallelism.

Factor-based strategy (adaptive parallelism via a factor f): To get a more aggressive, yet theoretically grounded, parallelism control, the authors propose a factor rule that effectively chooses the largest parallel batch whose worst-case marginal error keeps the theoretical bound satisfied. Empirically, factor decoding attains substantially higher throughput than the fixed-threshold variant while incurring only a small accuracy cost, making it attractive when latency matters.

Why these choices?
The paper's two main pragmatic constraints are (1) avoid retraining or architectural changes, and (2) preserve the quality advantages of diffusion LLMs while recovering autoregressive-style speed. The approximate KV cache exploits observed temporal stability of KV activations to cut redundant attention work without changing the model; the confidence/factor rules exploit a provable high-confidence regime to decide when independent marginal sampling is safe. Together they produce large, empirically validated speedups with minimal accuracy loss.

Limitations

KV Cache is Approximate: While the proposed caching method provides a significant throughput gain, it is still approximate and reducing the approximation would involve more frequent cache updates which would slow down the system, resulting in a tradeoff.

Scalability with Large Batch Sizes: While the proposed caching method excels at smaller batch sizes, it struggles to match the throughput of autoregressive models like LLaMA as batch sizes grow. LLaMA benefits significantly from large batch sizes by shifting from memory-bound to compute-bound performance.

Hyperparameter Trade-offs: The system requires careful tuning of the cache block size. Smaller blocks maximize accuracy but cause computational overhead due to frequent cache updates, while larger blocks improve speed but can diminish accuracy.

Unresolved Root Cause of Conditional Independence Assumption: While the confidence threshold mitigates the performance degradation due to the conditional independence assumption, it does not tackle the assumption head-on, which could be why there is still a performance degradation in accuracy.

Future Research Directions

Optimization for Longer Sequences: The authors note that while an upgraded model variant (LLaDA-1.5) showed great efficiency on shorter sequences, it experienced a mild efficiency regression at longer 512-token settings, suggesting that longer sequences require further optimization.

Bridging the Large-Batch Gap: Addressing the scalability limitations of diffusion models at larger batch sizes remains an ongoing challenge to make them fully competitive with AR models in high-throughput deployment settings

Better KV Cache Approximation: The authors of Fast-dLLM have published another paper, Fast-dLLM v2.0 where they replace the cache with a hierarchical cache with a single block level cache and a sub-block level DualCache.

Summary of Class Discussion

Q: In the figure in the paper where the authors show KV activations between adjacent inference steps are highly similar, did the ability to reuse only appear because of the large block size?
A: No, the ability to reuse the KV activations does not arise because of the large block size. The reuse is possible primarily because KV activations change very little between adjacent diffusion inference steps, and the block size simply determines how long the cache is reused before it is refreshed.

Q: In DualCache mode, why does caching the suffix help?
A: Caching the suffix in DualCache helps because even though those positions correspond to masked tokens, their attention representations (keys and values) are still computed and remain relatively stable across diffusion steps.

Q: Why would a token further in the sequence have a higher probability of being unmasked?
A: It is because of the tau-leaping where every position is sampled from a Poisson Distribution and is unmasked.

Q: Do they have to complete a block to get to the new one?
A: Yes a block needs to be completed because only then can the KV cache can be reused in a future block.

Q: Confidence level solves the dependency problem in AR models and maintains the dependency but does Fast-dLLM maintain the dependency between words?
A: The authors prove theoretically that their greedy parallel decoding yields the same results as the greedy sequential decoding in Theorem 1.

Q: Do you see the speed up trend happening for longer sequences permanently or will there be diminishing returns?
A. We will observe diminishing returns because the full attention mechanism will not be able to capture long range dependencies after a point.

Q: Is the structure for the multimodal experiments the same where the encoder converts it into a prompt for the diffusion LLM?
A: No, the structure does not change in any way.

Q: Will the frequent cache update of the approximate KV cache cause a throughput bottleneck?
A: Yes there will be a bottleneck if the KV cache is refreshed too frequently. There is a tradeoff between keeping the cache fresh and maintaining the throughput of the diffusion LLM.

Summary of ScaleFusion: Scalable Inference Of Spatial-Temporal Diffusion Transformers For High-Resolution Long Video Generation (Yang et al. 2025)

Problem and Motivation

Spatial-temporal diffusion transformers (ST-DiTs) combine diffusion-based frameworks with transformer architectures to capture spatial and temporal dependencies in video data. ST-DiTs use **spatio-temporal attention**, assigning tokens to both frames and spatial positions so that attention layers can learn dependencies like object motion, scene evolution, and multi-agent interaction. As SOTA video generation models improve (OpenAI Sora, CogVideoX, etc.), 1080p+ long-form video generation is becoming increasingly prevalent in industry, making efficient distributed inference a pressing need. However, attention computation in ST-DiTs **scales quadratically with both resolution and duration**, and even a single short 1080p clip requires tens of minutes of compute on a high-end GPU.

The root cause when scaling to multi-GPU machines is cross-machine communication overhead. ST-DiTs alternate between spatial and temporal attention layers, requiring all-to-all (A2A) communication between them to re-shard tensors. On a single machine this is negligible (~2-3%), since GPUs are connected via high-bandwidth NVLink/NVSwitch. Across machines, however, the system is bottlenecked by network interconnects (400 Gbps EFA), causing communication to account for 34% and 44% of end-to-end latency at 2 and 4 machines respectively (1.53x and 1.83x slowdowns relative to theoretical linear scaling). The natural fix is to overlap communication with computation, but ST-DiTs' rigid sequential dependency chain between A2A operations and attention layers makes this non-trivial. Existing SP techniques like DeepSpeed-Ulysses and DSP fail to break this chain, leaving communication overhead fully exposed. ScaleFusion addresses this gap with a **lossless, architecture-aware inference engine** that achieves **efficient computation-communication overlap** without sacrificing video quality.

Related Works

Sequence Parallelism Techniques: Efficient distributed inference over long sequences has been primarily addressed through sequence parallelism (SP) techniques. **DeepSpeed-Ulysses** uses all-to-all communication to transpose tensors between sequence and attention head dimensions. This minimizes communication volume but with no computation-communication overlap. It is also hard-bounded in parallelism by the number of attention heads, preventing it from scaling beyond 16 GPUs for the benchmarked ST-DiT. **RingAttention** pipelines key-value partitions across GPUs in a ring, achieving overlap but at the cost of much higher communication volume. This becomes a bottleneck in cross-machine settings with limited inter-machine bandwidth. **DSP**, the closest prior work and ScaleFusion's primary baseline, generalizes the Ulysses-style all-to-all approach across both spatial and temporal dimensions, making it well-suited to ST-DiTs. It reduces communication volume relative to

RingAttention and removes the head-count bottleneck, but still performs no computation-communication overlap, leaving 34% and 44% of latency exposed to communication at 2 and 4 machines respectively.

Pipeline Parallelism: Pipeline parallelism partitions model layers across GPUs and pipelines micro-batches through them. However, it operates along the batch dimension and does not address within-layer communication overhead. Nonetheless, it is orthogonal to ScaleFusion and can be applied in conjunction with it.

Lossy Optimizations for Diffusion Models: Methods like **DistriFusion** and **PipeFusion** exploit the iterative nature of diffusion sampling by reusing slightly stale activations from the previous denoising step to enable computation-communication overlap. While effective for image generation, output quality degrades as more GPUs are used. A larger fraction of activations become stale making these methods unsuitable for video generation at scale. Instead, ScaleFusion achieves overlap through a lossless structural insight, preserving output quality regardless of GPU count.

Solution Overview

ScaleFusion is motivated by a structural property of ST-DiTs that prior work had not exploited: **spatial-temporal independence**. When computing spatial attention, the temporal dimension is treated as a batch dimension, meaning each temporal token is processed independently. Similarly, when computing temporal attention, the spatial dimension is independent. This means the input tensor can be sliced along the independent dimension and each slice processed separately, with results later concatenated to produce an identical output. This breaks the monolithic A2A dependency into smaller, overlappable chunks and forms the foundation for ScaleFusion's two scheduling algorithms:

1. Intra-layer communication scheduling algorithm: Rather than executing each spatial or temporal layer on the full input tensor, ScaleFusion divides the input into $\mathbf{N_T}$ temporal slices (for spatial layers) or $\mathbf{N_S}$ spatial slices (for temporal layers) and pipelines their execution. While the computation for one slice is running, the A2A operation for the next slice is launched asynchronously on a separate CUDA stream. This overlaps the bulk of communication with computation, reducing the residual non-overlapped communication to approximately $\mathbf{Comm_T/N_T + Comm_S/N_S}$**.** Thus, only the first slice's A2A and the last slice's computation remain exposed. However, simply increasing the number of slices to minimize this residual is counterproductive. More slices means more fragmented CUDA kernel launches, which increases computation overhead and diminishes overall speedup. The authors find $\mathbf{N_T = N_S = 4}$ to be a robust default across workloads.

2. Inter-layer communication scheduling algorithm: ScaleFusion introduces this to address the irreducible non-overlap left by intra-layer scheduling. The idea is to lift part of the first slice's A2A operation into the previous layer's last-slice compute window. Each A2A can be further decomposed along the independent dimension into multiple partitions, and a subset of those partitions can be launched during the tail end of the previous layer's computation without adding to the critical execution path. Hyperparameters $\mathbf{L_T}$ and $\mathbf{L_S}$ control how many partitions are lifted from temporal and spatial layers respectively. This adds no computation overhead since no additional computation slices are introduced and only communication is

reordered. Combined with intra-layer scheduling, the non-overlapped communication is reduced to roughly $\mathbf{(Comm_T + Comm_S)/(N_S \times N_T)}$, achieving near-full overlap across all evaluated workloads.

A useful side effect of slicing A2A operations is reduced peak memory usage. Each A2A requires a temporary buffer to receive tensors from other GPUs. By releasing these buffers after each slice is complete rather than holding them all simultaneously, ScaleFusion achieves up to 2.33x peak memory reduction compared to DSP. This allows it to generate workloads like 4K 16-second videos that cause out-of-memory errors in prior methods. In end-to-end benchmarks on OpenSora v1.2, ScaleFusion achieves an average speedup of 1.36x (up to 1.58x) over DSP and strong scaling of 3.60x on 32 A100 GPUs, approaching theoretical linear scaling.

Limitations

The main problem with ScaleFusion is that it relies heavily on hardware infrastructure as the "communication-computation overlap" only works at its best when the network bandwidth is fast enough to keep up with the GPU's processing speed. On systems with standard, slow interconnects like Ethernet, the time it takes to communicate can exceed the computation time which can cause bottlenecks that make it impossible for the system to fully hide latency. Furthermore, the model is designed specifically for Spatial-Temporal Diffusion Transformers, which process space and time in alternating blocks. This means it can't be used with newer models that use a single, unified 3D-attention mechanism. Finally, the method needs more GPU memory to handle multiple data buffers for different "slices" of the video at the same time. This could cause memory problems while creating high-resolution sequences on hardware with limited VRAM.

Future Research Directions

Future ScaleFusion research may concentrate on developing dynamic scheduling techniques that change the total amount of "slices" and communication patterns in real-time. This would enable the system to maintain optimal efficiency even on heterogeneous hardware with varying network speeds. To further minimize the memory footprint and communication volume, researchers may consider combining this spatial-temporal partitioning with model compression methods like structural pruning or 4-bit quantization. Expanding the framework to accommodate unified 3D-attention architectures instead of merely decoupled spatial-temporal layers would increase its suitability for a greater variety of newly developed video generation models. Lastly, cross-step redundancy research holds great promise for advancing inference speeds toward real-time 1080p generation by reusing data from earlier diffusion timesteps to completely avoid redundant communication.

Summary of Class Discussion

Q: Does spatial-temporal independence exist by design in ST-DiTs, or is it something ScaleFusion introduces?
A: Spatial-temporal independence is an inherent structural property of ST-DiTs; it always existed in the architecture. ScaleFusion's contribution is not introducing this property, but being the first to recognize and exploit it as a mechanism for computation-communication overlap. Prior work like DSP was aware of

the spatial and temporal structure of ST-DiTs but did not leverage this independence to pipeline execution across slices.

Q: Is spatial-temporal independence unique to ST-DiTs, or can ScaleFusion generalize to other architectures?
A: Spatial-temporal independence is specific to ST-DiTs that use factored attention separate spatial and temporal attention layers. Models that use full 3D attention, where every token attends to every other token across both space and time simultaneously, do not have this property and ScaleFusion cannot be directly applied. So it would follow that any ScaleFusion could be applied to any model that fulfils the independence assumption.

Q: During spatial attention computation, don't patches have dependencies on surrounding patches, violating the independence assumption?
A: The independence ScaleFusion exploits is specifically along the temporal dimension, not the spatial dimension. During spatial attention, all spatial patches within a frame are fully present and attend to each other. The only thing absent is other frames, which is fine because spatial attention only operates within a single frame by design.

Q: Doesn't treating spatial and temporal dimensions as independent hurt video quality, leading to flickering or frame inconsistency?
A: No, this is what the temporal attention layer is for. Temporal attention enforces cross-frame consistency by attending to the same spatial patches across different frames. So independence is not assumed globally, just locally within each layer type.

Q: Why is ScaleFusion more memory efficient than prior approaches?
A: The memory savings are a direct byproduct of the intra- and inter-layer communication scheduling algorithms. By pipelining A2A operations across slices, each A2A only needs to allocate a temporary buffer for its individual slice rather than the full tensor at once. Once a slice's A2A completes, its buffer is immediately freed before the next slice begins. ScaleFusion's sliced pipelining therefore reduces peak memory usage proportionally to the number of slices.

Q: Why is RingAttention not included as a baseline despite being mentioned in the abstract?
A: It is not included as a baseline because DeepSpeed-Ulysses, which came after RingAttention, almost always outperforms it, particularly in cross-machine settings where RingAttention's higher communication volume is a critical disadvantage.

Q: Do different slices have different computation times, and how does the system manage these timing discrepancies?
A: In most cases each slice represents the same amount of computation since the attention math is uniform across the video. If dimensions don't divide evenly, the last slice may finish sooner, normally leaving the GPU idle. ScaleFusion addresses this through inter-layer scheduling. While the last slice is finishing, A2A operations for the next layer are already launched in the background, filling the gap with useful work.

Q: Does ScaleFusion allocate more computational resources to fast-moving objects or complex areas within a video?

A: No, ScaleFusion treats all parts of the video uniformly. It does not allocate more compute to complex regions like fast-moving objects. This is intentional: the entire scheduling strategy relies on each slice taking roughly the same amount of time to process, which is what enables predictable computation-communication overlap. Introducing content-adaptive compute would create variable slice timings, breaking the pipeline ScaleFusion depends on.